

Sabemi · Webhooks de Pagamento

Documentação técnica — Instalação · Configuração · Testes · Deploy



0 Problema

Um banco parceiro notifica pagamentos que liquidam seguros e parcelas de empréstimo. A notificação chega **repetida** — por erro de rede, reentrega ou retry do parceiro — e processar duas vezes significa somar o mesmo valor ao contrato.

Idempotência

Cada `id_transacao` processado uma única vez

Dois Backends

.NET e TypeScript, alternáveis em tempo real

Painel Sem Senha

Acesso que funciona entre aparelhos



Idempotência é do Banco, Não do Código



O problema do `if`

Verificar antes de inserir não garante nada sob concorrência: duas requisições passam pelo mesmo `if` juntas e ambas gravam.

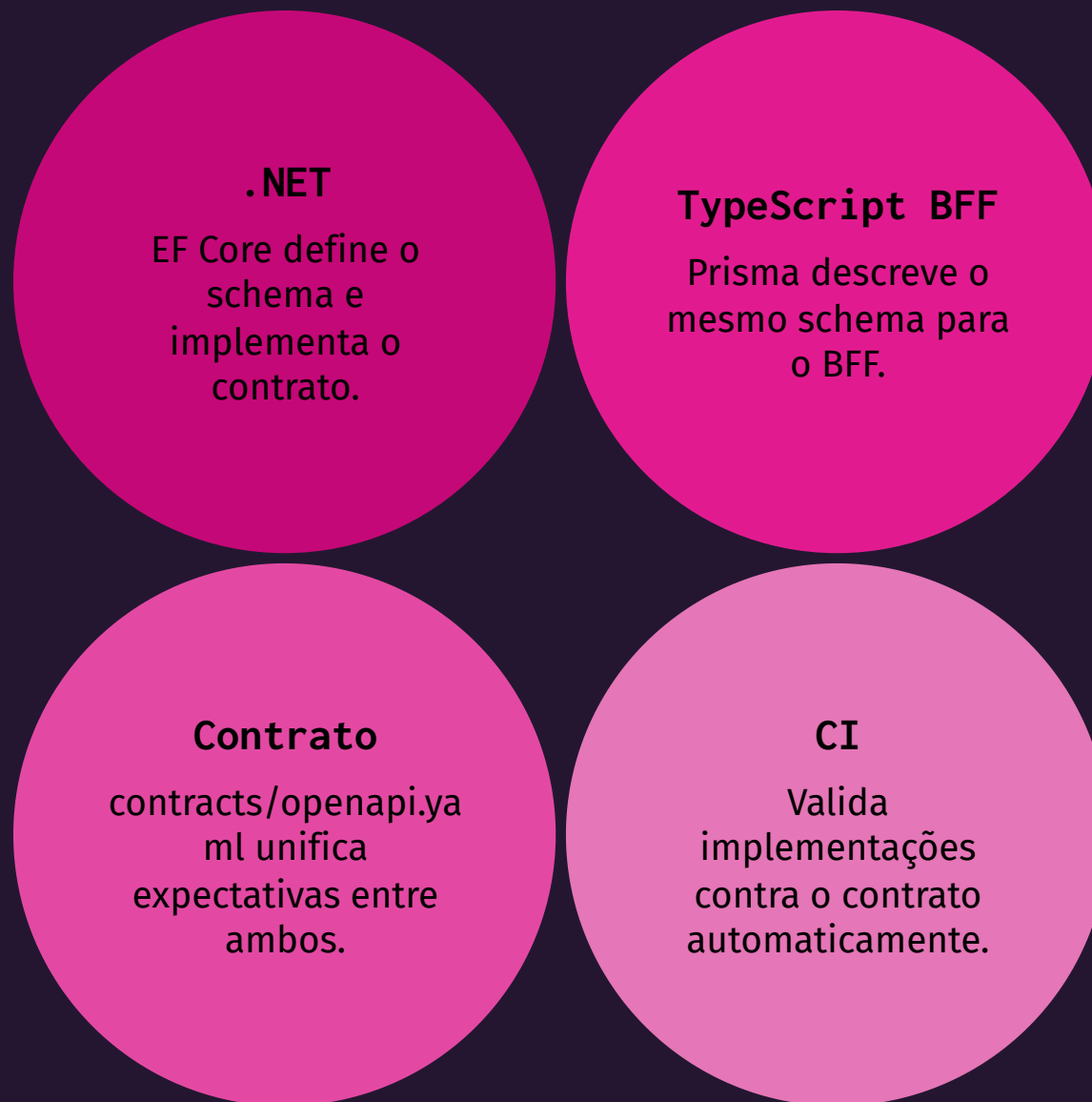
A garantia real

Índice único em `id_transacao` + `insert` otimista, capturando violação — 23505 no PostgreSQL, P2002 no Prisma. Verificado com 20 requisições simultâneas: um registro, uma consolidação.

Fila como tabela

Evento e job gravados na mesma transação. `FOR UPDATE SKIP LOCKED` permite múltiplas réplicas sem conflito. Worker que morre tem o item devolvido pelo *visibility timeout*. Falhas transitórias retentam; permanentes vão direto para **ERRO** com causa legível no painel.

Dois Backends Sobre um Contrato



contracts/openapi.yaml é implementado pelo .NET e pelo BFF em TypeScript, e validado no CI — é ele que torna a troca uma troca real, não uma simulação.

Schema compartilhado

Mesmas tabelas, usuários e fila. CI falha se EF Core e Prisma divergirem.

Troca transparente

Cookie lido no servidor; campo `backend` em `/health` vem da implementação que atendeu. Sessão assinada com o mesmo segredo — trocar de backend não desloga ninguém.

Testes de paridade

Acrescentar uma causa de falha em um só lado quebra o build.

Acesso e Segurança



Webhook Protegido

ApiKey no header + assinatura HMAC do corpo. Chave correta com corpo adulterado é recusada.



Magic Link Sem Senha

Link no desktop abre no celular; polling usa selector inofensivo; magic token nunca chega ao cliente. Sessão em cookie httpOnly — XSS não rouba token.



Rate Limit e CORS

Rate limit por IP, espera de reenvio contada na tabela compartilhada, CORS restrito à origem do painel.



Domínio .invalid

Nenhum teste sai para domínio real. Antes dessa regra, SMTP ativo gerou 26 *hard bounces* e endereços foram para blocklist — efeito acumulativo e irreversível.



A Prova

630

Testes reais

Atravessam a fiação real, não dublês do próprio código

236

.NET + PostgreSQL

Com banco de dados de verdade

342

Frontend + BFF

TypeScript

52

Ponta a ponta

Stack em containers

CI roda 4 jobs em paralelo com portão de cobertura, compila .NET tratando aviso como erro, valida OpenAPI e sobe a stack para teste de fumaça. Três serviços publicados via Dockerfiles, Supabase compartilhado — `git push` na main publica tudo.

⚠ **Armadilhas que não dão erro:** `?schema=` na URL vira nome de usuário no driver pg; Bootstrap `!important` vence o Tailwind; `sslmode=require` significa coisas diferentes no pg e no Npgsql; variável de ambiente pode valer só em metade do sistema. Todas produzem sistema no ar, respondendo 200, com resultado errado.



Limites Conhecidos

Saber onde está o limite vale mais do que fingir que ele não existe.

/metrics público

Expõe apenas contadores e latências, mas em produção seria fechado por rede.

Chave única do webhook

Rotacioná-la exige janela de indisponibilidade; o passo natural seria aceitar um conjunto de chaves.

Traces não correlacionados

traceparent não atravessa a fila; propagá-lo exigiria carregar contexto numa coluna e restaurá-lo no worker.

Métricas sem observação

Se a fila crescer sem parar, o painel mostra — desde que alguém esteja olhando.

Nenhum desses é obstáculo para o que o sistema faz hoje — são o que separa uma entrega sólida de uma operação em produção.